



Fig. 6: Log output in motes

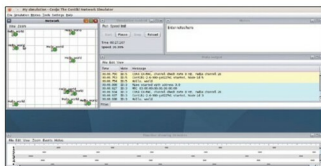


Fig. 7: Simulation window of Contiki

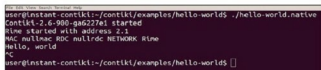


Fig. 8: Compilation using the terminal

which gives the name of the mote type as well as the Contiki application that the mote type will run. For this example, click the button on the right-hand side to choose the Contiki application and select `/home/user/contiki/examples/hello-world/hello-world.c`.

Then, click Compile.

Step 3. Once compiled without errors, click Create (Fig. 5).

Step 4. Now the screen will ask you to enter the number of motes to be created and their positions (random, ellipse, linear or manual).

In this example, ten motes are created. Click the Start button in the Simulation Control window and enable the mote's Log Output: `printf()` statements in the View menu of the Network window. The Network window shows the output Hello World in the sensors. Fig. 6 illustrates this.

This is a simple output of the Network window. If the real MicaZ motes are connected, Hello World will be displayed in the LCD panel of the sensor motes. The overall

output is shown in Fig. 7.

The output of the above Hello World application can also be run using the terminal.

To compile and test the program, go into the Hello World directory:

```
pradeep@localhost $ cd /home/user/contiki/examples/hello-world
pradeep@localhost $ make
```

This will compile the Hello World program in the native target, which causes the entire Contiki OS and Hello World application to be compiled into a single program that can be run by typing the following command (Fig. 8):

```
pradeep@localhost$ ./hello-world.native
```

This will print out the following text:

```
Contiki initiated, now starting process scheduling
Hello, world
```

The program will then appear to hang, and must be stopped by pressing Control + C.

Developing new modules

Contiki comes with numerous pre-built modules like IPv6, IPV6 UDP, Hello World, sensor nets, EEPROM, IRC, Ping, Ping-IPv6, etc. These modules can run with all the sensors irrespective of their make. Also, there are modules that run only on specific sensors. For example, the energy of a sky mote can be used only on sky motes and gives errors if run with other motes like Z1 or MicaZ.

Developers can build new modules for various sensor motes that can be used with different sensor BSPs using conventional C programming and then be deployed in the corresponding sensors.

Here is the C source code for the above Hello World application.

```
#include "contiki.h"
#include <stdio.h> /* For printf() */
/*-----*/
PROCESS(hello_world_process, "Hello world process");
AUTOSTART_PROCESSES(&hello_world_process);
/*-----*/
PROCESS_THREAD(hello_world_process, ev, data)
{
    PROCESS_BEGIN();
    printf("Hello, world\n");
    PROCESS_END();
}
```

The Internet of Things is an emerging technology that leads to concepts like smart cities, smart homes, etc. Implementing IoT is a real challenge but the Contiki OS can be of great help here. It can be very useful for deploying applications like automatic lighting systems in buildings, smart refrigerators, wearable computing systems, domestic power management for homes and offices, etc. ●

The author is a professor at VIT University, Chennai. He runs two websites <http://www.nsnam.com> and <http://www.pradeepkumar.org>. This article first appeared in September issue of Open Source For You